WELT 3 · DOPPELSTUNDEN 19 UND 20 (KÜR) · IDE-LEVEL 3.2 · DUNGEON-RENNEN

# W3·2 – Die Dungeon-Karte: Flood-Fill und Backtracking

Dateien: `AQ_W3_L3_Labyrinth_Geruest.txt` (Klasse Karte, Level 3.2) · `AQ_W3_DungeonRennen_Geruest.txt` (Nebel lüften, Dungeon-Rennen) · Im Lab: **Level 3.3** und **Level 3.4** · Teil 1 in DS 19, Teil 2 in DS 20 (Kür)

Die Kartografen haben das Labyrinth vermessen. Jede Karte ist ein Gitter aus Feldern: Wände, freie Gänge, ein Start und ein Ziel – und über allem liegt **Nebel**. Euer Spiel braucht drei Fähigkeiten: die Karte als **2D-Reihung** speichern und sicher auslesen, den Nebel mit Fackeln **lüften** (rekursiv, Feld für Feld) und den **Weg zum Ziel** finden – notfalls durch Ausprobieren und Umkehren.

Genau diese Werkzeuge kamen im Abitur vor: Spielfelder als 2D-Reihung mit Randfällen (2022) und eine rekursive Wegsuche im Labyrinth, deren Ablauf mit **Rekursionstiefe** dokumentiert werden musste (2024, Nachschreibtermin).

## Teil 1 – Die Karte und der Nebel (DS 19)

### Infoteil 1: Die Karte als 2D-Reihung

Eine Karte mit Zeilen und Spalten speichert man als **Reihung von Reihungen**: Jede Zeile ist selbst eine Reihung von Zeichen. Im Abitur heißt der Typ „Reihung von Reihung von Zeichen“, in Java `char[][]`.

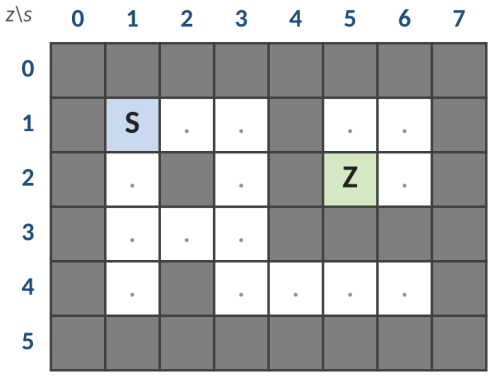



*Abbildung 1: Der Kerker – feld[z][s]*

- `char[][] feld = new char[6][8];` legt 6 Zeilen mit je 8 Spalten an.
- `feld[z][s]` ist das Feld in **Zeile z, Spalte s** – im Abitur immer **[Zeile][Spalte]**, Koordinaten geschrieben als (z|s).
- `feld.length` = Anzahl der **Zeilen**, `feld[0].length` = Anzahl der **Spalten**.
- Zeile 0 ist oben, Spalte 0 links. Gültige Indizes: 0 bis Länge – 1. Jeder andere Index führt zum Laufzeitfehler `ArrayIndexOutOfBoundsException`.
- Zeichen sind ein einfacher Typ: Vergleich mit `==`, z. B. `feld[z][s] == '#'` (Zeichenketten dagegen mit equals).

Zeichen auf der Karte: `#` Wand · `.` freier Gang (Nebel) · `S` Start · `Z` Ziel · `+` gelüftet · `*` Weg · `x` Sackgasse.

### Aufgabe 1: Koordinaten im Kerker

PAPIER ★

Die Karte in Abbildung 1 ist in der Variablen feld gespeichert.

**a)** Gib die Werte an. **[2 BE]**

| feld.length | feld[0].length | feld[2][5] | feld[4][3] | feld[3][4] | feld[1][6] |
|---|---|---|---|---|---|
| | | | | | |

**b)** Kreuze an, welche Zugriffe zu einem Laufzeitfehler führen: ☐ `feld[6][0]` ☐ `feld[0][8]` ☐ `feld[5][7]` ☐ `feld[-1][2]` **[1 BE]**

**c)** Gib die Koordinaten aller freien Felder ('.') in Spalte 3 an. **[1 BE]**

**d)** Ein Teammitglied vertauscht die Indizes und schreibt `feld[s][z]`. Gib an, was es für die Felder (2|5), (4|3) und (1|6) erhält, und erläutere, warum der Fehler auf quadratischen Karten oft unbemerkt bleibt. **[2 BE]**

### Infoteil 2: Durch die ganze Karte laufen

Wer **jedes** Feld besuchen muss (zählen, suchen, zeichnen), braucht zwei geschachtelte Zählschleifen: außen die Zeilen, innen die Spalten. Die Grenzen sind die Längen – nicht feste Zahlen.

```
for (int z = 0; z < feld.length; z++) {            // jede Zeile
    for (int s = 0; s < feld[z].length; s++) {     // jede Spalte dieser Zeile
        // bearbeite feld[z][s]
    }
}
```

Im Struktogramm: „für z von 0 bis Länge von feld − 1“ und darin „für s von 0 bis Länge von feld[0] − 1“. Für **eine** Zeile oder **eine** Spalte reicht eine einzige Schleife (die andere Koordinate bleibt fest).

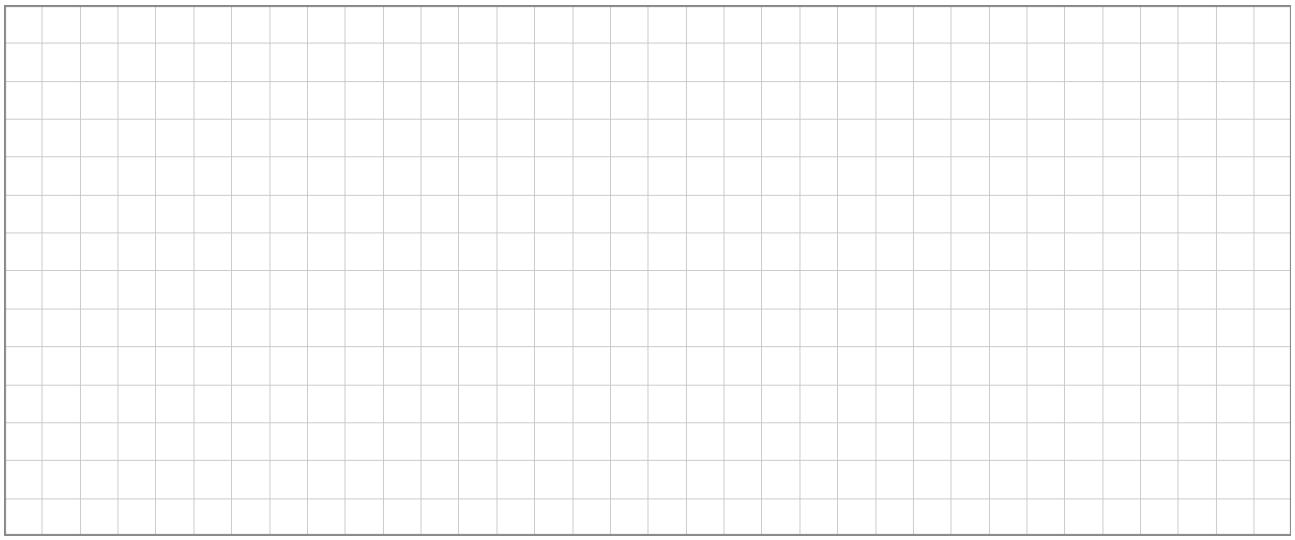
## Aufgabe 2: Rundgang durch die Karte

PAPIER ★★

Die folgenden Methoden gehören zur Klasse Karte mit dem Attribut `feld`.

**a)** Implementiere `anzahlInSpalte(s: Ganzzahl, c: Zeichen): Ganzzahl` in Java. Sie liefert, wie oft das Zeichen c in Spalte s vorkommt (0 ≤ s < Anzahl der Spalten). Gib die Rückgabe für den Kerker und den Aufruf `anzahlInSpalte(3, '.')` an. **[3 BE]**

**b)** Die Methode `istUmrandet(): Wahrheitswert` prüft, ob **alle Randfelder** der Karte Wände sind. Entwickle ein Struktogramm nach EH § 5 und gib an, was sie für den Kerker und für die Lichtung (Abbildung 3) liefert. **[4 BE]**



**c)** Gib die Ausgabe des folgenden Programmstücks für die Lichtung an (`print` gibt ohne Zeilenumbruch aus). **[2 BE]**

```
for (int s = 0; s < feld[0].length; s++) {
    for (int z = 0; z < feld.length; z++) {
        print(feld[z][s]);
    }
    println("");
}
```

## Infoteil 3: Nachbarn – und was am Rand passiert

**4er-Nachbarschaft**

| | | |
|---|---|---|
| – | 1 oben (z − 1 \| s) | – |
| 4 links (z \| s − 1) | (z \| s) | 2 rechts (z \| s + 1) |
| – | 3 unten (z + 1 \| s) | – |

**8er-Nachbarschaft**

| | | |
|---|---|---|
| (z − 1 \| s − 1) | (z − 1 \| s) | (z − 1 \| s + 1) |
| (z \| s − 1) | (z \| s) | (z \| s + 1) |
| (z + 1 \| s − 1) | (z + 1 \| s) | (z + 1 \| s + 1) |

*Abbildung 2: Nachbarn des Feldes (z|s) – in Welt 3 immer in der Reihenfolge oben, rechts, unten, links*

- **4er-Nachbarschaft:** nur waagerecht und senkrecht. In unseren Labyrinthen darf man nur so gehen. **8er-Nachbarschaft:** zusätzlich die vier Diagonalen.
- **Randfälle:** Am Rand und in den Ecken fehlen Nachbarn. Bevor du auf `feld[z – 1][s]` zugreifst, prüfst du, ob (z – 1|s) im Feld liegt: `istImFeld(z, s)` ist wahr für 0 ≤ z < feld.length und 0 ≤ s < feld[0].length.
- **Kurzschluss:** In `istImFeld(z, s) && feld[z][s] == '.'` wird der rechte Teil nur ausgewertet, wenn der linke wahr ist. Die Reihenfolge der Bedingungen ist deshalb **nicht** egal.
- Karten **mit** Randmauer verzeihen Fehler – die Karte „Lichtung“ (ohne Mauer) deckt sie auf.

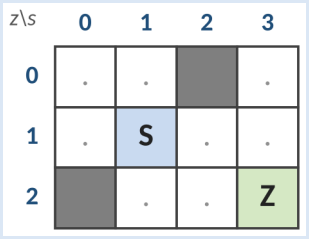



*Abbildung 3: Die Lichtung*

Die Lichtung aus dem Kartenbuch der Online-IDE hat 3 Zeilen und 4 Spalten und **keine** Randmauer. Begehbar sind die Felder mit '.', 'S' und 'Z'.

## Aufgabe 3: Wer ist mein Nachbar?

PAPIER ★★

**a)** Gib für die Lichtung die vier Nachbarn der Felder (0|1), (2|0) und (1|3) in der Reihenfolge oben, rechts, unten, links an. Markiere Nachbarn außerhalb der Karte mit „–“ und gib an, wie viele Nachbarn begehbar sind. **[3 BE]**

| Feld | oben | rechts | unten | links | begehbar |
|---|---|---|---|---|---|
| (0\|1) | | | | | |
| (2\|0) | | | | | |
| (1\|3) | | | | | |

**b)** Gib für den Kerker (Abbildung 1) die Anzahl der begehbaren Nachbarn (4er-Nachbarschaft) der Felder (1|2), (3|1), (4|6) und (2|5) an. **[2 BE]**

**c)** Auf einer Karte mit mindestens 3 Zeilen und 3 Spalten: Wie viele Nachbarn in der 8er-Nachbarschaft hat ein Eckfeld, ein Randfeld (keine Ecke) und ein inneres Feld? **[1 BE]**

## Infoteil 4: Nebel lüften – der rekursive Flood-Fill

Eine Fackel auf einem freien Feld erhellt das Feld – und gibt das Licht an ihre Nachbarn weiter, diese an ihre Nachbarn usw. So wird **jedes erreichbare** Feld gelüftet. Dieses Verfahren heißt **Flood-Fill** (Fluten). Hier die einfachste Form für eine Karte nur aus '#' und '.':

| **fackel(z: Ganzzahl, s: Ganzzahl): nichts** | |
|---|---|
| istImFeld(z, s) und feld[z][s] = '.' | |
| wahr | falsch |
| feld[z][s] ← '+' *// ZUERST markieren* | *// Basisfall: nichts tun* |
| fackel(z − 1, s) *// oben* | |
| fackel(z, s + 1) *// rechts* | |
| fackel(z + 1, s) *// unten* | |
| fackel(z, s − 1) *// links* | |

- **Basisfall:** außerhalb der Karte, Wand oder schon gelüftet → nichts tun.
- **Erst markieren, dann weitergehen:** Das Feld wird zu '+', **bevor** die Nachbarn beauftragt werden. Sonst würden sich zwei Nachbarn gegenseitig endlos aufrufen.
- Ein Aufruf **endet sofort**, wenn sein Feld nicht frei ist – auch solche Aufrufe zählen als Aufrufe.

**Beispiel:** fackel(0, 0) auf der kleinen Nische. Die Zahlen geben an, als wievieltes ein Feld markiert wurde, darunter die Rekursionstiefe des markierenden Aufrufs (Erstaufruf = Tiefe 0).

| z\s | 0 | 1 | 2 |
|---|---|---|---|
| 0 | . | . | |
| 1 | . | . | . |

*vorher*

| z\s | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1<br>Tiefe 0 | 2<br>Tiefe 1 | |
| 1 | 5<br>Tiefe 3 | 3<br>Tiefe 2 | 4<br>Tiefe 3 |

*nachher, mit Reihenfolge und Tiefe*

Ablauf: (0|0) wird markiert, beauftragt oben (außerhalb), dann rechts (0|1), Tiefe 1. Von dort: oben außerhalb, rechts Wand, unten (1|1), Tiefe 2. Von (1|1) aus: oben schon gelüftet, rechts (1|2), Tiefe 3 – fertig zurück; unten außerhalb; links (1|0), Tiefe 3. Insgesamt 21 Aufrufe von fackel für 5 markierte Felder.

## Aufgabe 4: Nebel lüften in der Wachstube

PAPIER ★★

Die Wachstube hat keine Randmauer. Es wird `fackel(0, 0)` aus Infoteil 4 aufgerufen.

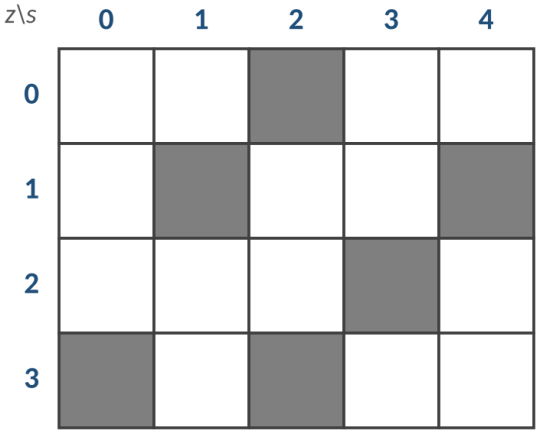



*Abbildung 4: Die Wachstube (weiß = '.', grau = '#')*

**a)** Trage in Abbildung 4 in jedes markierte Feld ein, als wievieltes es markiert wird, und darunter die Rekursionstiefe des markierenden Aufrufs. **[4 BE]**

**b)** Gib an, welche Felder im Nebel bleiben, und begründe. **[1 BE]**

**c)** Bestimme die Anzahl **aller** Aufrufe von fackel (auch die, die sofort enden). Begründe allgemein: Werden m Felder markiert, so gibt es $1 + 4 \cdot m$ Aufrufe. **[3 BE]**

**d)** Ein Teammitglied setzt die Zeile `feld[z][s] ← '+'` ans **Ende** des wahr-Zweigs. Erläutere am Beispiel der Felder (0|0) und (0|1), was dann passiert. **[2 BE]**

**e)** Ändert sich das **Ergebnis** (welche Felder gelüftet sind), wenn die Nachbarn in einer anderen Reihenfolge beauftragt werden? Ändert sich die **Nummerierung**? Begründe kurz. **[2 BE]**

## Aufgabe 5: Abi-Format: Wie lang ist der Gang?

PAPIER ★★★

Die Operation `gangLaenge(z: Ganzzahl, s: Ganzzahl, dz: Ganzzahl, ds: Ganzzahl): Ganzzahl` der Klasse Karte liefert die Anzahl der Felder, die man ab (z|s) **einschließlich** geradeaus in Richtung (dz|ds) durchläuft, bis eine Wand oder der Rand erreicht ist. Dabei ist (dz|ds) eine der vier Richtungen (−1|0), (0|1), (1|0) oder (0|−1). Liegt (z|s) außerhalb oder auf einer Wand, wird 0 geliefert. Beispiele für den Kerker: `gangLaenge(4, 3, 0, 1)` liefert 4, `gangLaenge(1, 6, 0, -1)` liefert 2.

**a)** Entwickeln Sie ein Struktogramm für die Operation gangLaenge. Schleifen dürfen nicht verwendet werden. **[4 BE]**

**b)** Dokumentieren Sie den Aufruf `gangLaenge(1, 1, 1, 0)` für den Kerker in einer Tabelle mit den Spalten Rekursionstiefe, Aufruf, Zeichen auf dem Feld und Rückgabe. **[3 BE]**

| Rekursionstiefe *) | Aufruf | Zeichen | Rückgabe |
|---|---|---|---|
| 0 | | | |
| | | | |
| | | | |
| | | | |
| | | | |

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

**c)** Begründen Sie, dass gangLaenge für jede Eingabe endet – auch auf der Lichtung ohne Randmauer. **[2 BE]**

## Aufgabe 6: Level 3.2 – Die Dungeon-Karte (Basis und Erweiterung)

IDE ★ – ★★

Arbeite in der Klasse **Karte** unterhalb der Linie „AB HIER ARBEITEST DU“, Testklasse `L32KarteTest`. Die Testkarten Höhle, Labyrinth, Kerker und Lichtung liefert die Klasse `Kartenbuch`.

| Stufe | Methoden (Signatur wie im Abitur) | Tests beginnen mit |
|---|---|---|
| ★ Basis | istImFeld(z: Ganzzahl, s: Ganzzahl): Wahrheitswert<br>istBegehbar(z: Ganzzahl, s: Ganzzahl): Wahrheitswert<br>anzahlWaende(): Ganzzahl | `basis_` |
| ★★ Erweiterung | anzahlBegehbareNachbarn(z: Ganzzahl, s: Ganzzahl): Ganzzahl<br>luften(z: Ganzzahl, s: Ganzzahl): nichts (REKURSIV)<br>anzahlGelueftet(): Ganzzahl | `erweiterung_` |

- `luften` arbeitet wie fackel, aber begehbar sind '.', 'S' **und** 'Z' – und S und Z müssen am Ende wieder als 'S' und 'Z' sichtbar sein. Überlege dir zuerst auf Papier, wie du S und Z während der Rekursion als besucht markierst.
- Die Lichtung hat keine Randmauer: Stürzt ein Test mit `ArrayIndexOutOfBoundsException` ab, fehlt eine Randprüfung. Meldet er StackOverflowError, wird zu spät (oder gar nicht) markiert.

**a)** **Nebel sehen:** Wer fertig ist, schreibt im Rennen-Gerüst `AQ_W3_DungeonRennen_Geruest.txt` die Methode `MeinSolver.luften` (Modus 1) und startet `Rennen.nebel(1)`, `(2)` und `(3)`. Die Konsole meldet „gelüftet“ und „Prüfungen“ (Aufrufe von istBegehbar). Prüfe den Zusammenhang aus Aufgabe 4 c).

| Rennkarte | gelüftet m | Prüfungen | 1 + 4 · m | korrekt? |
|---|---|---|---|---|
| 1 (klein) | | | | |
| 2 (mittel) | | | | |
| 3 (groß) | | | | |

### 🎯 Im Lab üben

- **Level 3.3 „Wachturm: 2D-Reihung und Flood-Fill“** – passt zu Teil 1, Aufgaben 1–5

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

# Teil 2 – Backtracking und das Dungeon-Rennen (DS 20, Kür)

Lüften zeigt, **welche** Felder erreichbar sind. Ein Held will aber wissen, **wie** er zum Ziel kommt. Theseus fand im Labyrinth des Minotaurus den Weg zurück, weil Ariadne ihm einen Faden mitgab: Er wickelte ihn ab, solange er vorwärtsging – und zog ihn wieder ein, wenn er in einer Sackgasse stand.

## Infoteil 5: Backtracking – ausprobieren und Schritt zurücknehmen

- **Idee:** Gehe einen Schritt auf ein freies Feld und markiere es vorläufig als Weg (`'*'`). Versuche von dort aus nacheinander die Nachbarn (oben, rechts, unten, links). Führt einer zum Ziel: fertig. Führt **keiner** zum Ziel, war der Schritt falsch – **nimm ihn zurück** (das Feld wird Sackgasse 'x') und melde falsch an den Aufrufer, der dann seinen nächsten Nachbarn probiert.
- „oder“ wird von links nach rechts ausgewertet und **bricht beim ersten wahr ab**: Ist der Weg nach oben erfolgreich, werden rechts, unten und links gar nicht mehr versucht.
- Der Aufrufstapel **ist** der Ariadnefaden: Er enthält genau die Felder des aktuellen Weges von S bis zum aktuellen Feld.

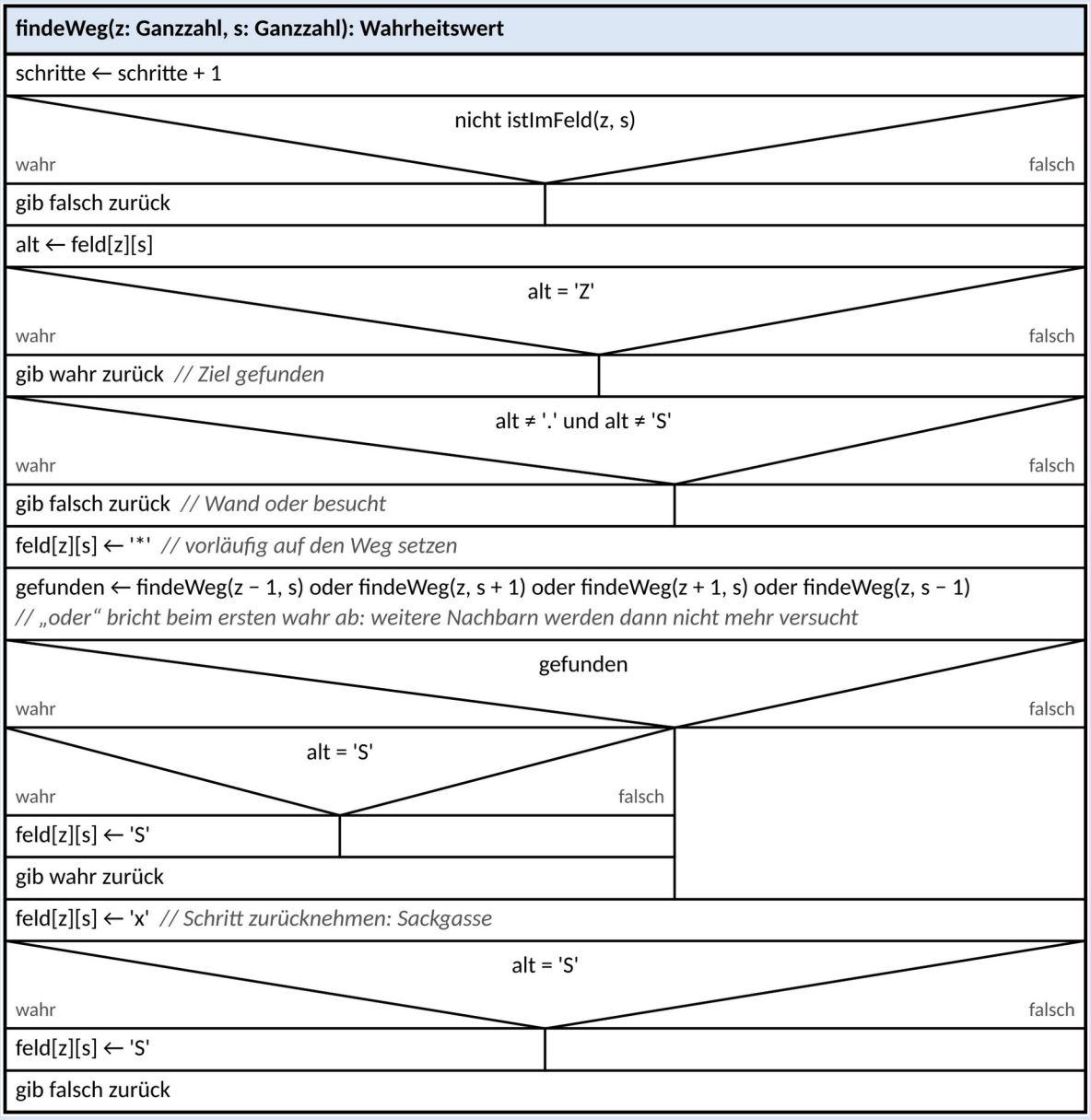



*Abbildung 5: findeWeg der Klasse Karte (Level 3.2 Experte); schritte zählt alle Aufrufe*

## Aufgabe 7: Der Ariadnefaden im Gewölbe

PAPIER ★★

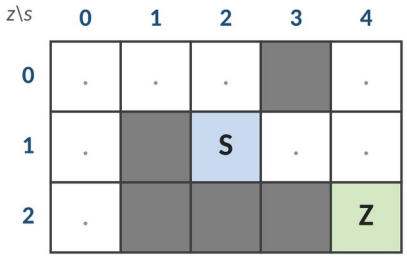



*Abbildung 6: Das Gewölbe (keine Randmauer)*

Im Gewölbe wird `findeWeg(1, 2)` aus Abbildung 5 aufgerufen, Start ist S auf (1|2).
Die Tabelle hat das Format aus dem Abitur 2024: **eine Zeile für jeden Aufruf, der ein Feld betritt** (also '.' oder 'S' zu `'*'` macht) **oder das Ziel erreicht**. „Richtung“ ist der Schritt vom aufrufenden Feld aus, „Weg“ die Folge der Felder von S bis zum betretenen Feld.

**a)** Dokumentieren Sie den Ablauf in der Tabelle. **[6 BE]**

| Rekursionstiefe *) | Richtung | Weg |
|---|---|---|
| 0 | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

*In der ersten Zeile steht: Rekursionstiefe 0 · Richtung – (Aufruf) · Weg (1|2).*

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

**b)** Zeichnen Sie die Karte nach dem Ende des Erstaufrufs (mit `'*'`, 'x' und den nie betretenen Feldern). **[2 BE]**

**c)** Erläutern Sie, wie sich die Belegung von feld durch die Anweisung `feld[z][s] ← 'x'` verändert, und warum ohne diese Anweisung nach dem Ende ein falscher Weg auf der Karte stünde. **[3 BE]**

## Aufgabe 8: Sackgassen auf der Lichtung

PAPIER ★★

| z\s | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | x | x | | x |
| 1 | x | S | * | * |
| 2 | | . | . | Z |

*Abbildung 7: Lichtung nach findeWeg(1, 1)*

Der Test `experte_findeWeg_rand` erwartet nach `findeWeg(1, 1)` auf der Lichtung die Karte in Abbildung 7 (vorher: Abbildung 3).

**a)** Erläutere ohne vollständige Tabelle, warum (0|1), (0|0) und (1|0) zu 'x' wurden, obwohl der Held in der Nähe des Starts war. In welcher Reihenfolge wurden die drei Felder betreten, in welcher zurückgenommen? **[3 BE]**

**b)** Begründe, warum (2|1) und (2|2) '.' bleiben, obwohl sie erreichbar sind. **[1 BE]**

**c)** Ein Teammitglied ersetzt `feld[z][s] ← 'x'` durch `feld[z][s] ← '.'`. Beurteile: Endet findeWeg trotzdem immer? Was ändert sich für die Karte und für die Anzahl der Aufrufe? **[2 BE]**

**d)** „Backtracking findet immer den kürzesten Weg.“ Widerlege die Aussage mit einer selbst gezeichneten kleinen Karte (höchstens 2 × 3 Felder). **[2 BE]**

## Aufgabe 9: Abi-Format: Wie lang ist der Weg?

PAPIER ★★★

Die Operation `wegLaenge(z: Ganzzahl, s: Ganzzahl): Ganzzahl` soll wie findeWeg per Backtracking (oben, rechts, unten, links) einen Weg von (z|s) zum Ziel suchen, aber statt eines Wahrheitswerts die **Anzahl der Schritte** dieses Weges liefern: 0, wenn (z|s) selbst das Ziel ist; −1, wenn kein Weg gefunden wird. Die Markierungen `'*'` und 'x' bleiben wie bei findeWeg; schritte wird nicht gezählt. Beispiel: Für die Lichtung liefert `wegLaenge(1, 1)` den Wert 3.

**a)** Verändern Sie das Struktogramm aus Abbildung 5 zu einem Struktogramm für wegLaenge. **[5 BE]**

**b)** Geben Sie die Rückgaben von `wegLaenge` für den Kerker (Start (1|1)) und das Gewölbe (Start (1|2)) an. **[1 BE]**

**c)** Begründen Sie, dass die von wegLaenge gelieferte Zahl im Allgemeinen nicht die Länge eines kürzesten Weges ist. **[2 BE]**

## Aufgabe 10: Level 3.2 Experte – findeWeg

IDE ★★★

Implementiere `findeWeg(z: Ganzzahl, s: Ganzzahl): Wahrheitswert` in der Klasse Karte nach Abbildung 5. Die Tests mit `experte_` prüfen die Endkarten von Höhle, Labyrinth, Kerker und Lichtung **und** den Zähler `schritte`.

- **Zähl-Regel:** `schritte++` ist die **erste** Anweisung – bei jedem Aufruf, auch außerhalb der Karte und auf Wänden. Die Nachbarn werden **ohne Vorprüfung** aufgerufen, Reihenfolge oben, rechts, unten, links, Abbruch beim ersten true (`||`).
- Nimmst du den Schritt nicht zurück oder setzt S nicht wieder auf 'S', zeigt die Testmeldung die ganze erwartete und tatsächliche Karte – vergleiche Feld für Feld.

### Das Dungeon-Rennen – die Regeln (Kopf von AQ_W3_DungeonRennen_Geruest.txt)

Wessen Held findet mit den **wenigsten Schritten** vom Start S zum Ziel Z? Es tritt eure Methode `MeinSolver.suche(z, s): Wahrheitswert` an. Gezählt wird nicht die Zeit, sondern **jedes betretene Feld**.

| Befehl | Wirkung |
|---|---|
| `Rennen.zeilen()`, `Rennen.spalten()` | Größe der Karte |
| `Rennen.istBegehbar(z, s)` | wahr, wenn (z\|s) in der Karte liegt, keine Wand ist und noch nicht betreten wurde |
| `Rennen.istZiel(z, s)` | wahr, wenn (z\|s) das Ziel Z ist |
| `Rennen.zielZeile()`, `Rennen.zielSpalte()` | der **Kompass**: Wo liegt Z? |
| `Rennen.betrete(z, s)` | betritt das Feld (gold) – zählt **einen Schritt** |
| `Rennen.verlasse(z, s)` | verlässt das Feld als Sackgasse (rot), ein Feld zurück – zählt **nicht** |

1. Drei feste **Rennkarten** (klein 7 × 11, mittel 11 × 15, groß 15 × 21), danach drei unbekannte **Kontrollkarten** gleicher Größe. Gestartet wird mit `Rennen.starten()`.
2. **Wertung:** Summe der Schritte auf den drei Rennkarten – **weniger ist besser**. Auch der Schritt auf S und der auf Z zählen.
3. **Gültig** nur, wenn auf allen sechs Karten suche wahr liefert und ein zusammenhängender Weg aus betretenen Feldern von S nach Z führt.
4. **Bewegung:** zuerst S betreten, danach nur ein Nachbarfeld des Standorts (oben, rechts, unten, links) – keine Sprünge, keine Wände, kein Feld zweimal. `verlasse` nur für den Standort.
5. **Nebel-Regel:** `istBegehbar` und `istZiel` nur für den Standort und seine vier Nachbarn – weiter sieht der Held nicht. Nur der Kompass zeigt immer zum Ziel.
6. **Verboten:** feste Wege oder Sonderregeln für bekannte Karten, `Math.random`, die Klasse Rennen verändern, andere Rennen-Befehle während der Suche.
7. Am Ende steht in der Konsole `RENNEN-ERGEBNIS: Schritte = …, gueltig = ja`. Jedes Team meldet **ein** Ergebnis; die Lehrkraft trägt es in die Ergebnistabelle an der Tafel ein (Einheit Schritte, weniger ist besser). Das Rennen ist ein **Team-Wettbewerb im Unterricht** – es fließt nicht in die Lab-Punkte ein. Das beste gültige Team stellt seine Strategie vor.

## Aufgabe 11: Das Dungeon-Rennen

IDE ★★ – ★★★ Team-Wettbewerb

**a)** **Standard zuerst:** Übertrage findeWeg auf die Rennen-API: nicht begehbar → falsch; betreten; Ziel → wahr; Nachbarn oben, rechts, unten, links rekursiv versuchen; keiner erfolgreich → verlassen, falsch. Teste mit `Rennen.training(1)`, `(2)`, `(3)` und `(0)` (Zufallskarte). Verfolge die Animation mit langsamem Tempo.

**b)** **Strategie-Phase:** Spart Schritte! Ideen: andere Reihenfolge der Richtungen, „Richtung Ziel zuerst“ mit dem Kompass, Regeln bei Gleichstand. Führt Buch:

| Strategie | Karte 1 | Karte 2 | Karte 3 | Summe | gültig? |
|---|---|---|---|---|---|
| Standard: oben, rechts, unten, links | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

c) **Abgabe:** `Rennen.starten()` ausführen, Ergebniszeile der Lehrkraft zeigen. Erläutert in einem Satz, warum eure Strategie auf einer der Karten **schlechter** sein kann als das Standardverfahren.

### Für Schnelle: Diagonal und mit Kompass

- **8er-Nachbarschaft:** Im Labyrinth des Kartenbuchs liegt unten rechts eine Kammer, die nur **über Eck** mit dem Gang verbunden ist. Schreibe `luefte8(z, s)` mit allen acht Nachbarn (zwei Zählschleifen für dz und ds von −1 bis 1, (0|0) auslassen). Welche Felder werden zusätzlich gelüftet?
- **Wie tief geht es?** Sage voraus, welche größte Rekursionstiefe fackel(0, 0) auf einer **leeren** 4 × 4-Karte erreicht. Prüfe mit einer Rekursionstabelle oder in der IDE. Was bedeutet das für sehr große Karten?
- **Kompass-Heuristik:** Sortiere die vier Richtungen nach dem Manhattan-Abstand |z − zielZeile| + |s − zielSpalte| des Nachbarfeldes. Findest du eine Rennkarte, auf der die Heuristik verliert?

### 🎯 Im Lab üben

- **Level 3.4 „Kür: Ariadnefaden – Backtracking“** – passt zu Teil 2, Aufgaben 7–9

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit Kurscode, Name und deinem persönlichen Code.

### Abi-Check 2D-Reihung und Backtracking

- ☐ Indizes immer [Zeile][Spalte]; Zeilen: Länge von feld, Spalten: Länge von feld[0].
- ☐ Vor **jedem** Feldzugriff prüfen, ob (z|s) im Feld liegt – die Prüfung steht links vom Zugriff.
- ☐ Rekursion auf der Karte: Basisfall (Rand, Wand, besucht) zuerst; **erst markieren, dann weitergehen**.
- ☐ Backtracking: Erfolg sofort weitermelden; ohne Erfolg den Schritt zurücknehmen und falsch liefern.
- ☐ Tabellen: eine Zeile pro (betretendem) Aufruf, Erstaufruf = Tiefe 0, Geschwisteraufrufe haben **dieselbe** Tiefe.